

Stratégie d'optimisation de la base de données OLAP

Le principal enjeu concernant la base de données OLAP de Stripe est le traitement de millions de transactions. Ce document décrit les stratégies mises en place pour optimiser les performances de l'analyse sur un tel volume de données.

1. Optimisation des requêtes et des jointures

La première chose à faire est de réduire le volume total scanné pour accéder aux données. Pour ce faire, les tables ont été réfléchies pour faciliter les jointures à grande échelle. Des index ont donc été créés sur les clés étrangères de fact_transaction (avec customer_key, merchant_key, geography_key, time_key et payment_key).

Aussi, il est important de pouvoir sélectionner facilement la plage de temps nécessaire à l'analyse. C'est pour cette raison que la dimension dim_time décompose aussi précisément le temps, par exemple en jour, mois et années. Ainsi, la table fact_transaction est partitionnée par année et mois via la colonne time_key. Cela permet à la base de données de ne scanner que la partition correspondant à la période recherchée.

Enfin, pour optimiser la puissance de calcul quand on veut faire une sélection dans une sélection, il est préférable d'éviter de le faire sous forme de sous-requête. En effet, celles-ci vont s'exécuter autant de fois qu'il y a de lignes. Il faut alors les remplacer par des jointures, ou par des Common table expressions, qui permettent plus de rapidité.

2. Vues matérialisées

Des vues matérialisées ont également été créées. Ce sont des requêtes dont le résultat est stocké afin de faciliter l'analyse. Elles permettent de répondre rapidement aux principales questions métier. Elles sont mises à jour chaque nuit par l'intermédiaire de requêtes enregistrées dans le fichier "queries_olap.sql".

La première vue matérialisée est summary_daily_revenue, qui répond à la question concernant les performances de Stripe sur une journée donnée. Chaque colonne correspond à la réponse à une question :

Quel est le revenu total d'une journée ?

Combien de transactions ont eu lieu en une journée ?

Quel est le montant moyen de ses transactions ?

Combien d'entre elles ont échoué ?

Combien de fraudes ont été détectées durant cette journée ?

La deuxième vue matérialisée est summary_fraud_by_country, qui concerne les fraudes par pays. Les colonnes répondent aux questions :

Combien de fraudes a-t-il eu lieu en tout dans ce pays ?

Quel est le montant que cela représente ?

Quel est le taux de fraude par pays ?

La dernière vue matérialisée est `summary_merchant_performance`, qui s'intéresse cette fois-ci aux marchands. Les colonnes répondent aux questions suivantes :

Quel est le revenu total d'un marchand en particulier ?

Quel est le nombre total de ses transactions ?

Quel est son taux de remboursement ?

Quel est son taux de fraude ?

Ainsi, le temps de réponse aux requêtes est réduit par l'intermédiaire des index et du partitionnement temporel. Le remplacement des sous-requêtes par les jointures permet d'optimiser la puissance de calcul et les vues matérialisées permettent des analyses rapides sur les millions de lignes de la base de données de Stripe.